LIBRARY MANAGEMENT SYSTEM | SQL

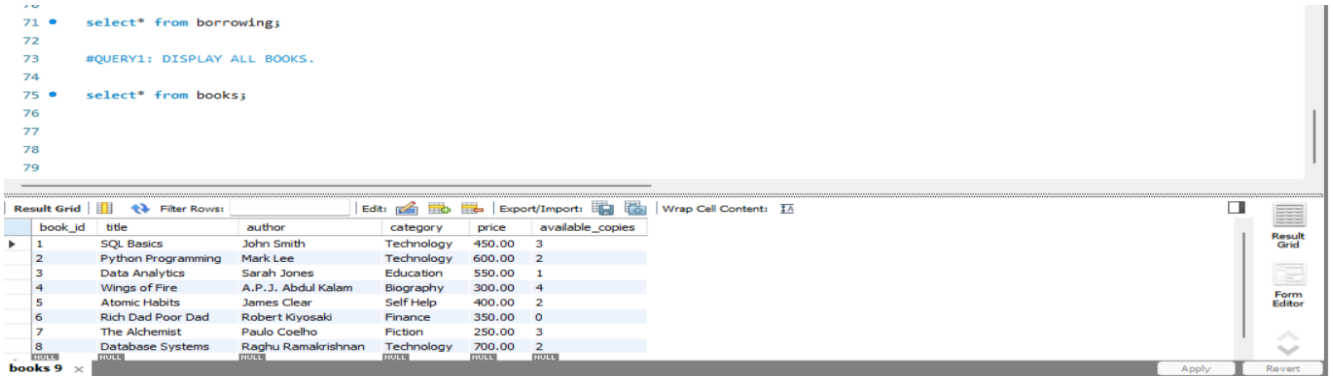
SQL Project Documentation | MySQL | Data Analyst Portfolio

Project overview: This project uses a relational database to manage books, members, and borrowing transactions. The 20 queries below demonstrate filtering, aggregation, date handling, joins, inventory analysis, and member activity analysis.

QUERY 1: Display All Books

SQL QUERY

```
SELECT *
FROM Books;
```



Explanation

SELECT * displays every column and every record in the Books table.

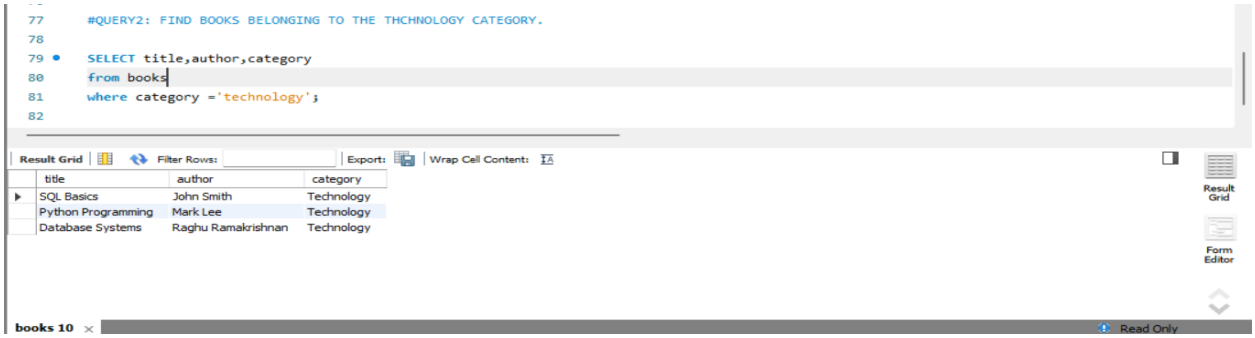
Business-world use

This gives staff a complete view of the book catalogue, including titles, authors, categories, prices, and available copies.

QUERY 2: Find Books in the Technology Category

SQL QUERY

```
SELECT title, author, category
FROM Books
WHERE category = 'Technology';
```



Explanation

WHERE filters records based on a condition. This query returns only books classified as Technology.

Business-world use

A library can review its technology collection and decide whether it needs to add or update titles in this subject area.

QUERY 3: Find Books Costing More Than ₹400

SQL QUERY

```
SELECT title, author, price
FROM Books
WHERE price > 400;
```



Explanation

The condition price > 400 returns books whose listed price is greater than ₹400.

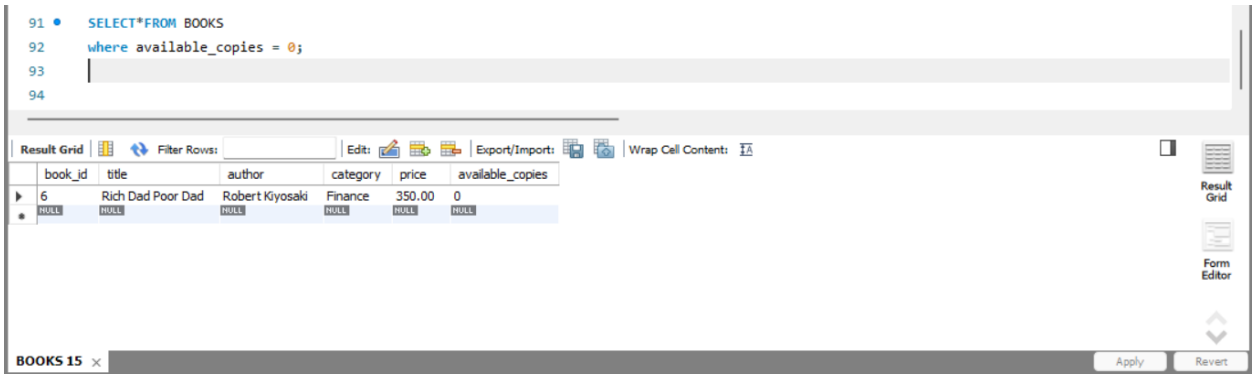
Business-world use

This helps staff identify higher-priced titles and plan purchasing budgets.

QUERY 4: Find Books That Are Currently Unavailable

SQL QUERY

```
SELECT *
FROM Books
WHERE available_copies = 0;
```



Explanation

The condition available_copies = 0 returns books with no copies currently available.

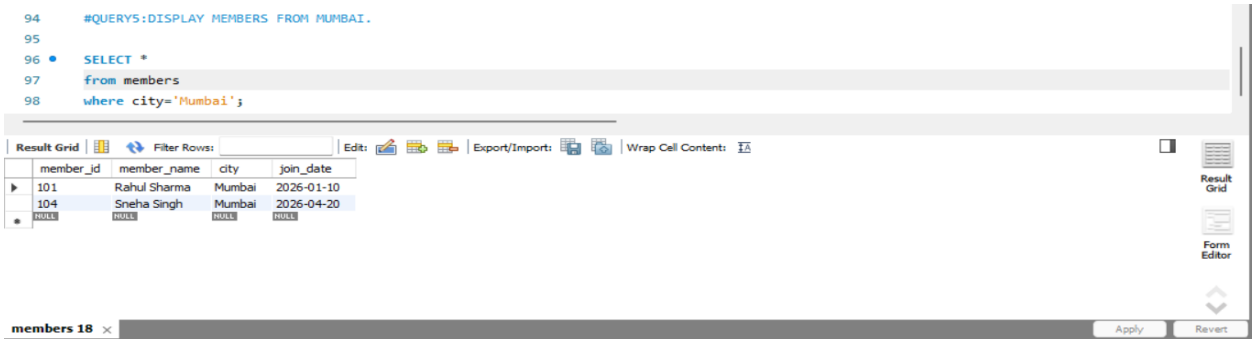
Business-world use

Staff can identify out-of-stock titles and consider buying additional copies.

QUERY 5: Display Members from Mumbai

SQL QUERY

```
SELECT *
FROM Members
WHERE city = 'Mumbai';
```



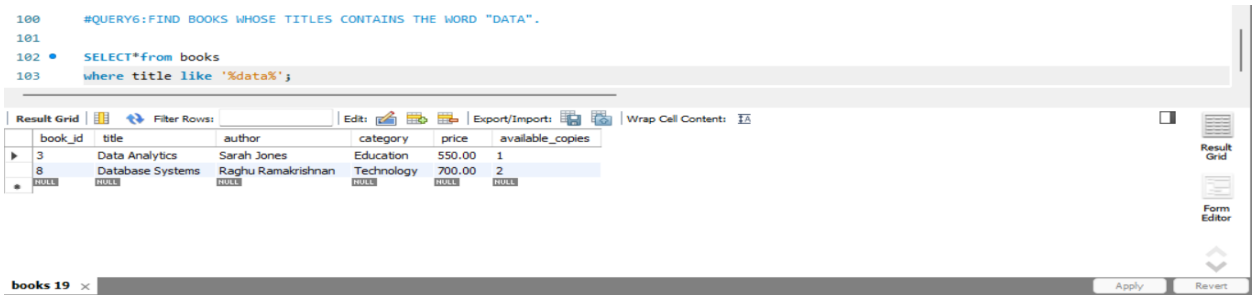
Explanation The WHERE condition filters members whose city is Mumbai.

Business-world use The library can understand its membership by location and plan relevant local communications or services.

QUERY 6: Find Book Titles Containing the Word “Data”

SQL QUERY

```
SELECT *
FROM Books
WHERE title LIKE '%data%';
```



Explanation

LIKE searches for a text pattern. The % wildcards allow any characters before or after “data”.

Business-world use

Staff can find relevant books using a keyword without knowing the full title.

QUERY 7: Count the Total Number of Book Records

SQL QUERY

```
SELECT COUNT(*) AS total_books
FROM Books;
```



Explanation

COUNT(*) counts the number of rows in Books. The alias total_books gives the result column a clear name.

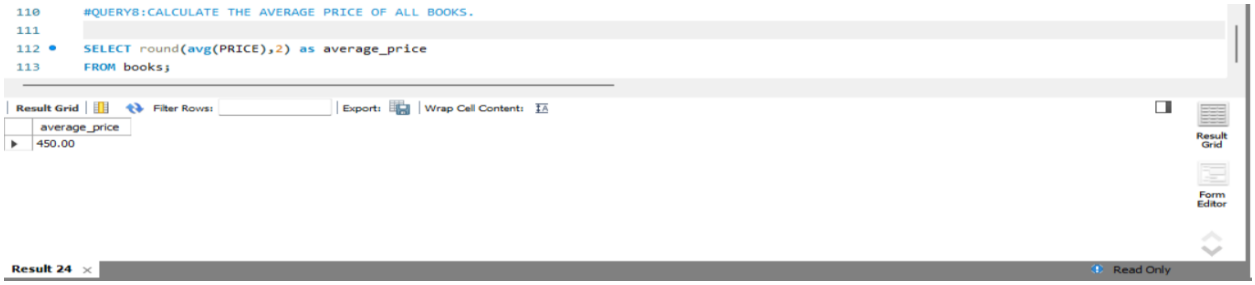
Business-world use

This provides a quick count of the catalogue records maintained in the database.

QUERY 8: Calculate the Average Price of Books

SQL QUERY

```
SELECT ROUND(AVG(price), 2) AS average_price
FROM Books;
```



Explanation

AVG(price) calculates the mean price, and ROUND(..., 2) rounds it to two decimal places.

Business-world use

The average price helps staff understand the cost level of the collection and estimate future purchasing budgets.

QUERY 9: Count Books in Each Category

SQL QUERY

```
SELECT category, COUNT(*) AS number_of_books
FROM Books
GROUP BY category
ORDER BY number_of_books DESC;
```



Explanation

GROUP BY creates one group per category, COUNT(*) counts records in each group, and ORDER BY sorts categories from largest count to smallest.

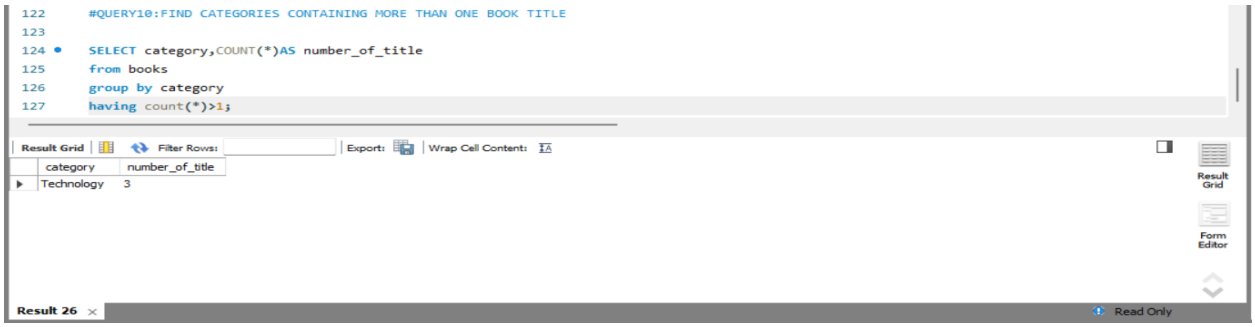
Business-world use

This shows how the collection is distributed across subjects and helps guide future purchases.

QUERY 10: Find Categories with More Than One Book Title

SQL QUERY

```
SELECT category, COUNT(*) AS number_of_titles
FROM Books
GROUP BY category
HAVING COUNT(*) > 1;
```



Explanation

GROUP BY groups records by category. HAVING filters the grouped results and keeps categories with more than one title.

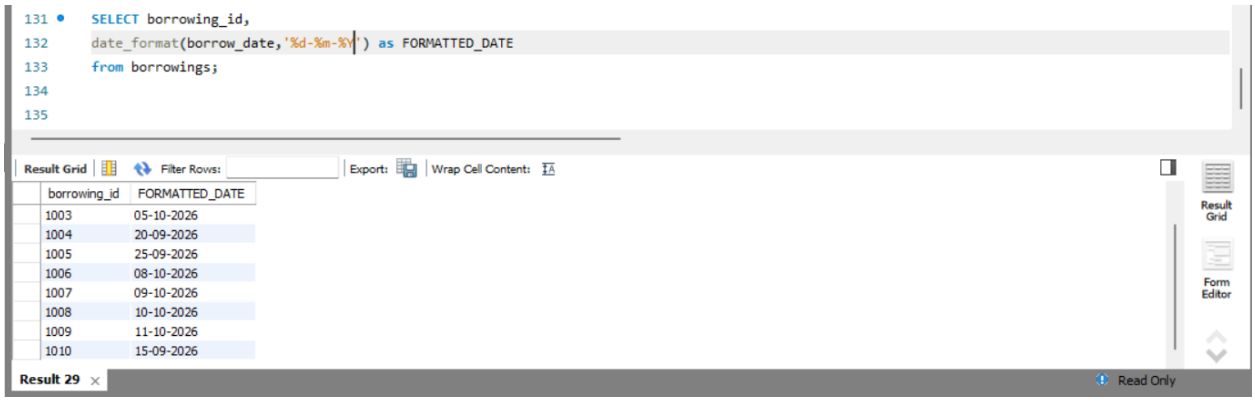
Business-world use

This identifies subject areas with a larger collection and can inform collection planning.

QUERY 11: Display Borrowing Dates in DD-MM-YYYY Format

SQL QUERY

```
SELECT borrowing_id,
       DATE_FORMAT(borrow_date, '%d-%m-%Y') AS formatted_date
FROM Borrowings;
```



Explanation

DATE_FORMAT converts each borrow_date to day-month-year format. %d is the day, %m the month, and %Y the four-digit year.

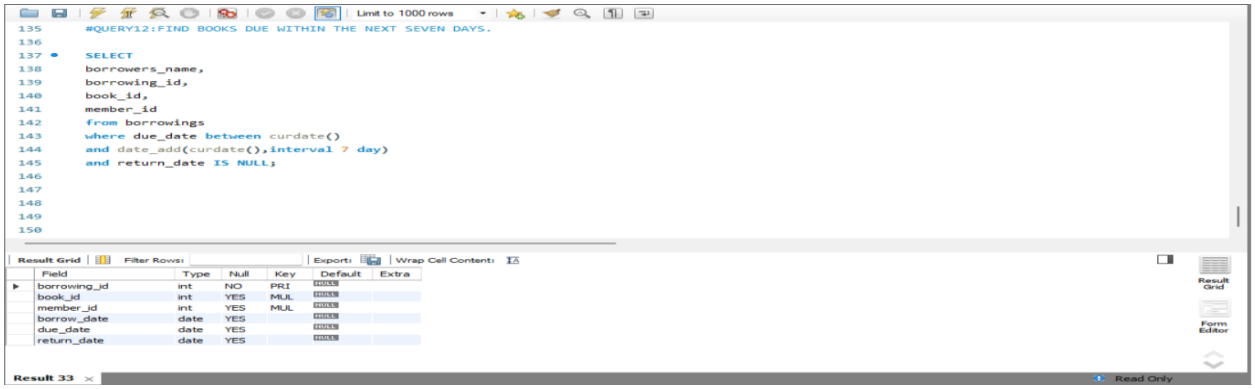
Business-world use

Consistent date formatting makes borrowing records easier to read in reports and presentations.

QUERY 12: Find Unreturned Books Due Within the Next Seven Days

SQL QUERY

```
SELECT borrowing_id, book_id, member_id, due_date
FROM Borrowings
WHERE due_date BETWEEN CURDATE()
                   AND DATE_ADD(CURDATE(), INTERVAL 7 DAY)
  AND return_date IS NULL;
```



Explanation

CURDATE() returns today's date. DATE_ADD calculates the date seven days ahead. BETWEEN includes both endpoints, while return_date IS NULL keeps only unreturned books.

Business-world use

Staff can identify upcoming due dates and remind members before books become overdue.

QUERY 13: Calculate How Many Days a Book Is Overdue

SQL QUERY

```
SELECT borrowing_id,
       due_date,
       DATEDIFF(CURDATE(), due_date) AS days_overdue
FROM Borrowings
WHERE due_date < CURDATE()
  AND return_date IS NULL;
```

```sql
#QUERY13: CALCULATE THE NUMBER OF DAYS A BOOK IS OVERDUE.

SELECT
borrowing_id,
due_date,
datediff(CURDATE(),due_date) as days_overdue
from borrowings
where due_date < curdate()
and return_date is null;
```

borrowing_id	due_date	days_overdue
1002	2026-10-12	2
1004	2026-09-30	14

Result 38 — Read Only

Explanation

DATEDIFF calculates the number of days between today and the due date. The filters keep past-due records that have not been returned.

Business-world use

This helps staff prioritize overdue returns and follow up with members.

QUERY 14: Calculate Overdue Fines at ₹5 per Day

SQL QUERY

```sql
SELECT m.member_name,
       b.title,
       br.due_date,
       DATEDIFF(CURDATE(), br.due_date) AS days_overdue,
       DATEDIFF(CURDATE(), br.due_date) * 5 AS fine_amount
FROM Borrowings AS br
JOIN Members AS m
  ON br.member_id = m.member_id
JOIN Books AS b
  ON br.book_id = b.book_id
WHERE br.return_date IS NULL
  AND br.due_date < CURDATE();
```

```sql
#QUERY14:CALCULATE OVERDUE FINES AT Rs.5 PER DAY

SELECT
m.member_name,
b.title,
br.due_date,
datediff(curdate(),br.due_date) as days_overdue,
datediff(curdate(),br.due_date)*5 as fine_amount
from borrowings br
join members m
on br.member_id = m.member_id
join books b
on br.book_id = b.book_id
where br.return_date is null
and br.due_date < curdate();
```

member_name	title	due_date	days_overdue	fine_amount
Priya Patel	Python Programming	2026-10-12	2	10
Amit Verma	Rich Dad Poor Dad	2026-09-30	14	70

Result 37 — Read Only

Explanation

The joins retrieve the member name and book title. DATEDIFF calculates overdue days, and multiplying by 5 calculates a sample fine at ₹5 per day. Only unreturned, overdue records are included.

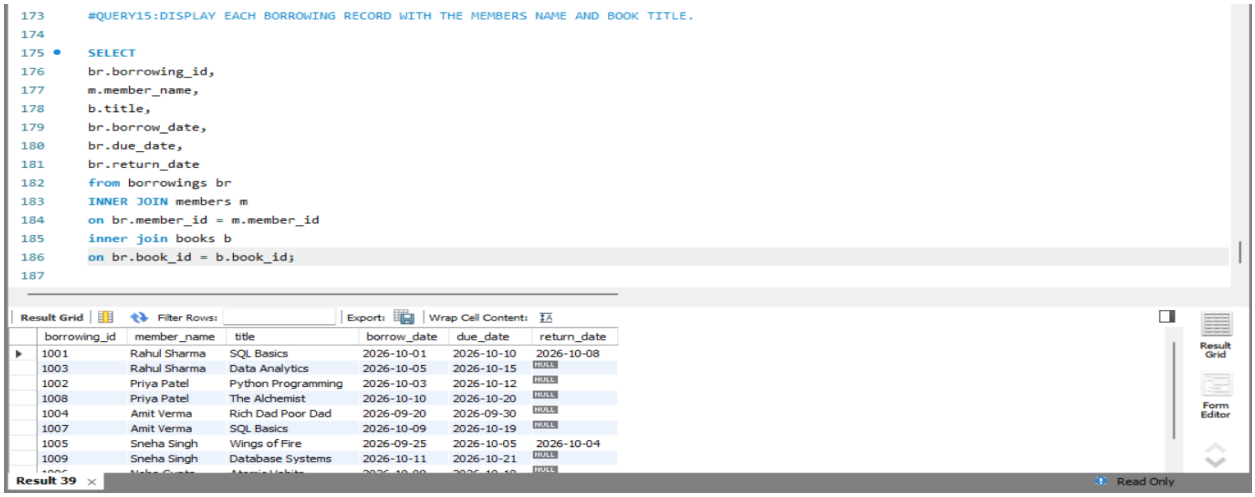
Business-world use

This helps staff estimate outstanding fines and contact members about late returns. The ₹5 rate is an example rule used for this project.

QUERY 15: Display Borrowing Records with Member Names and Book Titles

SQL QUERY

```
SELECT br.borrowing_id,
       m.member_name,
       b.title,
       br.borrow_date,
       br.due_date,
       br.return_date
FROM Borrowings AS br
INNER JOIN Members AS m
  ON br.member_id = m.member_id
INNER JOIN Books AS b
  ON br.book_id = b.book_id;
```



```
#QUERY15:DISPLAY EACH BORROWING RECORD WITH THE MEMBERS NAME AND BOOK TITLE.

SELECT
br.borrowing_id,
m.member_name,
b.title,
br.borrow_date,
br.due_date,
br.return_date
from borrowings br
INNER JOIN members m
on br.member_id = m.member_id
inner join books b
on br.book_id = b.book_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

borrowing_id	member_name	title	borrow_date	due_date	return_date
1001	Rahul Sharma	SQL Basics	2026-10-01	2026-10-10	2026-10-08
1003	Rahul Sharma	Data Analytics	2026-10-05	2026-10-15	NULL
1002	Priya Patel	Python Programming	2026-10-03	2026-10-12	NULL
1008	Priya Patel	The Alchemist	2026-10-10	2026-10-20	NULL
1004	Amit Verma	Rich Dad Poor Dad	2026-09-20	2026-09-30	NULL
1007	Amit Verma	SQL Basics	2026-10-09	2026-10-19	NULL
1005	Sneha Singh	Wings of Fire	2026-09-25	2026-10-05	2026-10-04
1009	Sneha Singh	Database Systems	2026-10-11	2026-10-21	NULL

Result 39 | Read Only | Result Grid | Form Editor

Explanation

INNER JOIN combines related rows from Borrowings, Members, and Books using their matching IDs.

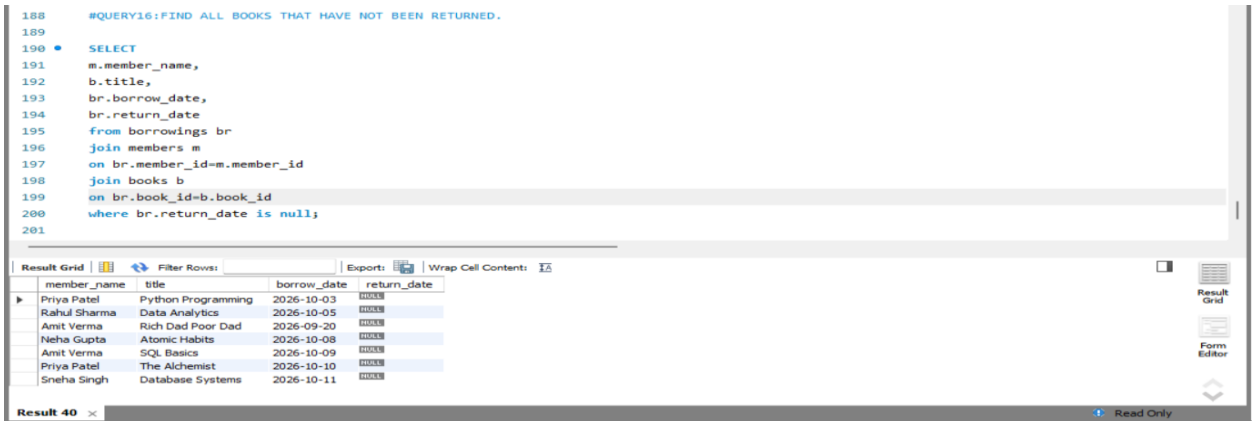
Business-world use

Staff can see who borrowed each book and review the borrow, due, and return dates in one report.

QUERY 16: Find All Books That Have Not Been Returned

SQL QUERY

```
SELECT m.member_name,
       b.title,
       br.borrow_date,
       br.return_date
FROM Borrowings AS br
JOIN Members AS m
  ON br.member_id = m.member_id
JOIN Books AS b
  ON br.book_id = b.book_id
WHERE br.return_date IS NULL;
```



Explanation

The joins show the member and book details. return_date IS NULL identifies borrowing records that have not been marked as returned.

Business-world use

This helps staff monitor outstanding loans and follow up where necessary.

QUERY 17: Count How Many Times Each Book Has Been Borrowed

SQL QUERY

```
SELECT book_id,
       COUNT(*) AS times_borrowed
FROM Borrowings
GROUP BY book_id
ORDER BY times_borrowed DESC;
```

```
202     #QUERY17:COUNT HOW MANY TIMES EACH BOOK HAS BEEN BORROWED.
203
204 •   select
205     book_id,
206     count(*) as times_borrowed
207     from borrowings
208     group by book_id
209     order by times_borrowed desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Result Grid | Form Editor

book_id	times_borrowed
1	2
2	2
3	1
4	1
5	1
6	1
7	1
8	1

Result 42 | Read Only

Explanation

GROUP BY groups borrowing records by book_id, and COUNT(*) counts the records for each book. Results are sorted from most to least borrowed.

Business-world use

This helps identify borrowing patterns and provides evidence for future book-purchasing decisions.

QUERY 18: Find the Two Most Frequently Borrowed Books

SQL QUERY

```
SELECT b.title,
       COUNT(br.borrowing_id) AS total_borrowings
FROM Books AS b
JOIN Borrowings AS br
  ON b.book_id = br.book_id
GROUP BY b.book_id, b.title
ORDER BY total_borrowings DESC
LIMIT 2;
```

```
213 •   SELECT
214     b.title,
215     count(borrowing_id) as total_borrowings
216     from books b
217     join borrowings br
218     on b.book_id = br.book_id
219     group by b.book_id, b.title
220     order by total_borrowings desc
221     limit 2;
222
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows: | Result Grid | Form Editor

title	total_borrowings
SQL Basics	2
Python Programming	2

Result 44 | Read Only

Explanation

The query joins books to borrowing records, counts borrowings per title, sorts in descending order, and returns the top two results.

Business-world use

Popular titles may need additional copies. Ties can affect which titles appear when several books have the same count.

QUERY 19: Calculate the Total Value of Available Book Inventory

SQL QUERY

```
SELECT SUM(price * available_copies) AS inventory_value
FROM Books;
```



Explanation

For each title, price multiplied by available_copies estimates the value of its available stock. SUM adds these values together.

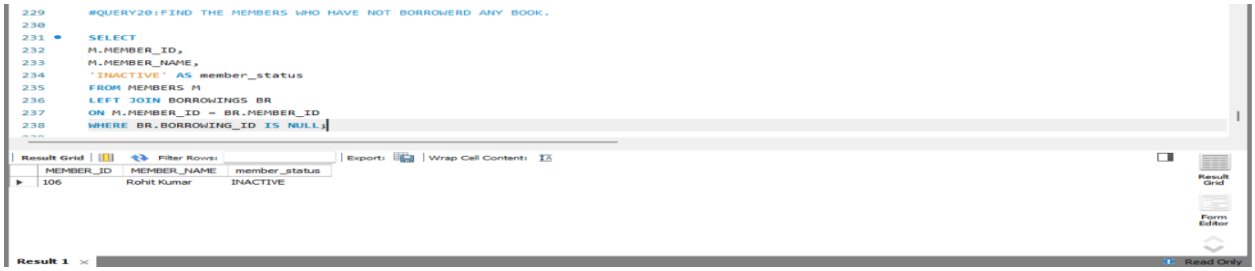
Business-world use

This estimates the value of available inventory and supports stock and budget planning.

QUERY 20: Find Members Who Have Never Borrowed a Book

SQL QUERY

```
SELECT m.member_id,
       m.member_name,
       'INACTIVE' AS member_status
FROM Members AS m
LEFT JOIN Borrowings AS br
  ON m.member_id = br.member_id
WHERE br.borrowing_id IS NULL;
```



Explanation

LEFT JOIN keeps every member, including those without a matching borrowing record. The IS NULL condition selects members with no borrowing history.

Business-world use

The library can identify members who have not used the borrowing service and consider outreach or promotions. “INACTIVE” is a project label, not a formal status from the database.